

BÁO CÁO BÀI TẬP LAB 03

Đánh giá hiệu quả Agent với ba mẫu thiết kế

ReAct Plan and Execute Hybrid

Thông tin	Nội dung
Chủ đề	Agent đặt vé máy bay có kiểm soát
Nền tảng	LangChain LangGraph Pydantic
Mô hình	OpenAI compatible chat model
Sinh viên	
Ngày báo cáo	28 tháng 9 năm 2026

Mục tiêu của bài là xây dựng, kiểm soát và so sánh ba cách tổ chức agent trên cùng một bài toán đặt vé, thay vì chỉ kiểm tra khả năng sinh văn bản của mô hình.

Tóm tắt báo cáo

Bài tập xây dựng một hệ thống agent đặt vé máy bay gồm model thật, hai tool nghiệp vụ, ba kiến trúc điều phối và một harness an toàn độc lập. Ba kiến trúc được so sánh là ReAct, Plan and Execute và Hybrid. Điểm quan trọng nhất của thiết kế cuối không nằm ở việc model chọn được tool, mà ở việc tách quyền quyết định khỏi quyền thực thi side effect: agent chỉ đề xuất đặt vé, còn FlightHarness xác minh kết quả search, yêu cầu người dùng phê duyệt, phát hiện lặp và chỉ gọi book_flight thật sau user submit.

Bộ benchmark trong lab03.py dùng cùng model, tool, dữ liệu và chính sách an toàn cho cả ba thiết kế. Ba tình huống đánh giá buộc agent phải loại chuyển hết chỗ, so sánh nhiều lựa chọn và dừng an toàn khi không có chuyến đúng ngày. Kết quả chạy model thật đạt 9/9 lượt: ReAct nhanh nhất, Hybrid đứng thứ hai về thời gian trung bình, còn Plan and Execute chậm hơn do có nhiều vòng planner, executor và replanner.

Nội dung báo cáo

1. Bài toán và mục tiêu thiết kế
2. Cấu trúc dự án và vai trò từng file
3. Dữ liệu và các tool nghiệp vụ
4. Model thật và structured output
5. Agent ReAct
6. Agent Plan and Execute
7. Agent Hybrid
8. FlightHarness và middleware an toàn
9. Luồng user approval trước khi đặt vé
10. Benchmark trong lab03.py
11. Kết quả kiểm thử và đánh giá
12. Hạn chế và hướng phát triển

1 Bài toán và mục tiêu thiết kế

Hệ thống nhận yêu cầu tự nhiên như tìm và đặt chuyến bay rẻ nhất còn chỗ. Agent phải tìm dữ liệu bằng search_flight_info, đọc observation, chọn một chuyến hợp lệ, đề xuất book_flight và dừng để chờ người dùng xác nhận. Hệ thống không được đặt vé chỉ vì model đã sinh ra một tool call.

Các yêu cầu kỹ thuật chính gồm:

- Dùng model thật thông qua ChatOpenAI và cấu hình trong file .env.
- Xây dựng ba luồng agent khác nhau nhưng dùng chung model và tool.
- Bảo toàn tool observation để planner hoặc replanner có căn cứ sửa kế hoạch.
- Không cho model trực tiếp thực hiện side effect đặt vé.
- Phát hiện hành động lặp, giới hạn số lần gọi tool và ghi lịch sử đầy đủ.
- So sánh hiệu quả bằng cùng một tập kịch bản và cùng tiêu chí đánh giá.

2 Cấu trúc dự án

File	Vai trò
lab03.py	Chạy benchmark, tạo scenario, thu metrics và in báo cáo so sánh.
lib tools_datve.py	Chứa dữ liệu tĩnh, hàm search_flight_info và book_flight.
lib model_react.py	Dựng ReAct agent bằng create_agent.
lib model_pe.py	Dựng graph Planner Executor Replanner.
lib model_hybrid.py	Dựng graph cấp cao với ReAct sub-agent ở node executor.
lib harness.py	Thực thi tool thật, lưu observation, approval gate, loop detector và middleware.
requirements.txt	Khai báo LangChain, LangGraph gián tiếp, dotenv và langchain-openai.

3 Dữ liệu và tool nghiệp vụ

3.1 Dữ liệu chuyến bay

VE_MAY_BAY là database tĩnh dùng để kết quả có thể tái hiện. Mỗi bản ghi có Airline, Departure, Arrival, Date, Hour, Price và State. Dữ liệu được mở rộng nhằm tạo tình huống cần suy luận thay vì luôn có một đáp án hiển nhiên.

Kịch bản	Dữ liệu nổi bật	Điều cần suy luận
Cơ bản	Hà Nội đến Đà Nẵng ngày 01 06	Vietjet và Bamboo rẻ hơn nhưng full; Vietnam Airlines mới available.
Chọn lọc	TP HCM đến Nha Trang ngày 01 07	Vietjet 180 full; Bamboo 240 là chuyến available rẻ nhất.
Không có ngày	Hà Nội đến Huế ngày 01 07	Không có kết quả đúng ngày; chỉ có dữ liệu ngày 02 07.

3.2 Tool tìm chuyến

```
def search_flight_info(
    departure: str,
    arrival: str,
    date: str,
    hour: str = None,
) -> list:
```

Tool chuẩn hóa chuỗi đầu vào rồi lọc đúng điểm đi, điểm đến, ngày và giờ tùy chọn. Đây là tool chỉ đọc nên có thể chạy trước approval gate.

3.3 Tool đặt vé

```
def book_flight(
    airline: str,
    departure: str,
    arrival: str,
    date: str,
    hour: str,
) -> dict:
```

Tool trả success khi tìm thấy chuyến available và trả error kèm hint khi chuyển hết chỗ hoặc không tồn tại. Trong kiến trúc cuối, raw book_flight không được đưa trực tiếp cho agent.

4 Model thật và structured output

Model được khởi tạo từ OPENAI_MODEL, OPENAI_API_KEY và OPENAI_BASE_URL. temperature bằng 0 nhằm giảm dao động khi model lập kế hoạch và chọn arguments cho tool.

```
model = ChatOpenAI(
    model=model_name,
    api_key=api_key,
    base_url=base_url,
    temperature=0,
)
```

Endpoint OpenAI compatible được sử dụng không trả structured function call ổn định cho with_structured_output. Vì vậy P&E và Hybrid dùng _invoke_structured: prompt yêu cầu duy nhất một JSON object theo JSON Schema, sau đó Pydantic kiểm tra bằng model_validate_json. Nếu parse thất bại, lỗi chứa raw response để dễ chẩn đoán.

Schema	Ý nghĩa
Plan hoặc HybridPlan	Danh sách các bước hoặc mục tiêu cấp cao.
ToolInstruction	Tên tool và dictionary arguments của bước P&E hiện tại.
ReplanDecision	Quyết định continue hoặc finish, plan còn lại, response và reason.
HybridReplanDecision	Quyết định sau khi đọc cả raw tool observations của ReAct executor.

5 Thiết kế ReAct

ReAct thực hiện Reason và Act xen kẽ. Mỗi vòng, model đọc message history và quyết định trả lời cuối hoặc sinh tool_calls. LangChain create_agent dựng sẵn một compiled LangGraph cho vòng lặp này.

5.1 Hàm tạo agent

Hàm create_react_agent nhận bốn thành phần: model kiểu BaseChatModel, danh sách BaseTool, system_prompt và middleware. Bên trong hàm, create_agent được gọi với list tools và list middleware. Giá trị trả về không phải một câu trả lời mà là CompiledStateGraph; chương trình có thể gọi invoke để lấy kết quả cuối hoặc stream để xem update của từng node.

```
return create_agent(
    model=model,
```

```

tools=list(tools),
system_prompt=system_prompt,
middleware=list(middleware),
)

```

5.2 Dòng dữ liệu khi chạy

1. main tạo ChatOpenAI từ biến môi trường rồi tạo FlightHarness bằng hai raw function search_flight_info và book_flight.
2. harness.langchain_tools tạo hai StructuredTool proxy. Tool tên book_flight thực tế trỏ tới propose_booking, không trỏ thẳng tới raw book_flight.
3. create_react_agent nhận proxy tools và HarnessGuardMiddleware. Human message được đưa vào state messages.
4. Model trả AIMessage. Nếu AIMessage có tool_calls, LangChain gọi proxy tool tương ứng; nếu không có tool_calls, graph kết thúc.
5. Kết quả proxy tool trở thành ToolMessage và được thêm vào messages. Model đọc lại toàn bộ history ở vòng kế tiếp.
6. Khi propose_booking trả approval_required, model phải hỏi người dùng hoặc middleware chặn tool call tiếp theo và kết thúc graph.
7. Sau graph, main đọc input y hoặc n. Chỉ harness.submit_booking True mới gọi raw book_flight.

5.3 Cách in trace

main dùng agent.stream với stream_mode updates. Mỗi update thường là dictionary có tên node và phần state thay đổi. Vì middleware đôi khi trả update None, code kiểm tra data is None trước khi gọi data.get. Việc kiểm tra này tránh lỗi AttributeError và giúp trace vẫn tiếp tục qua các node model, tools và middleware.

Ưu điểm của ReAct là linh hoạt và ít code orchestration. Nhược điểm là model quyết định cục bộ từng bước nên dễ retry hoặc đi lòng vòng nếu observation nghèo. Harness và middleware vì thế đặc biệt quan trọng.

6 Thiết kế Plan and Execute

P&E tách lập kế hoạch khỏi thực thi. Planner tạo danh sách bước; executor chỉ chọn đúng một tool cho bước hiện tại; replanner đọc observation và quyết định tiếp tục, sửa kế hoạch hoặc kết thúc.

6.1 State và reducer

State của graph gồm user_request, plan, past_steps và response. past_steps dùng reducer operator.add nên mỗi observation mới được nối vào lịch sử thay vì ghi đè.

```

class PlanExecuteState(TypedDict):
    user_request: str
    plan: list[str]
    past_steps: Annotated[list[tuple[str, str]], operator.add]
    response: str

```

6.2 Planner node

planner nhận state user_request và gọi _invoke_structured với schema Plan. Hàm _invoke_structured chèn JSON Schema vào SystemMessage, gọi model.invoke, lấy response.content, loại bỏ markdown fence nếu có,

tìm JSON object và dùng `Plan.model_validate_json` để kiểm tra. Planner trả dictionary chỉ chứa plan; LangGraph ghép phần cập nhật này vào state hiện có.

6.3 Executor node

executor lấy state plan phần tử đầu làm step hiện tại. Model được yêu cầu trả `ToolInstruction` gồm `tool_name` và `arguments`. `tools_by_name` tra tool hợp lệ; tên không tồn tại tạo observation `unknown_tool`. Nếu tên hợp lệ, `tool.invoke` được gọi trong try except. Mọi exception được chuyển thành observation có status `error` thay vì làm graph sập.

Executor nhận tool proxy của harness. Vì vậy dòng `tool.invoke` không gọi raw tool trực tiếp: `search_flight_info` đi qua `FlightHarness.search_flight_info`, còn `book_flight` đi qua `FlightHarness.propose_booking`. Observation sau đó được JSON hóa và nối vào `past_steps` dưới dạng cặp step observation.

6.4 Replanner và routing

replanner đọc `user_request`, plan hiện tại và toàn bộ `past_steps`. `ReplanDecision.action` bằng `finish` sẽ xóa plan và ghi response; action bằng `continue` sẽ thay plan bằng `remaining_steps`. Hàm `route_after_replan` kiểm tra response: có response thì đi END, chưa có response thì quay lại executor. Prompt yêu cầu finish ngay khi observation có `approval_required` để chờ user submit, không gọi `book_flight` proxy lần nữa.

```
graph.add_edge(START, "planner")
graph.add_edge("planner", "executor")
graph.add_edge("executor", "replanner")
graph.add_conditional_edges(
    "replanner",
    route_after_replan,
    {"continue": "executor", "finish": END},
)
```

7 Thiết kế Hybrid

Hybrid kết hợp kế hoạch cấp cao của P&E với khả năng phản ứng nhiều vòng của ReAct. Planner tạo mục tiêu, nhưng mỗi mục tiêu được giao cho một ReAct sub-agent có thể gọi tool nhiều lần trước khi trả kết quả về replanner.

7.1 ReAct executor bên trong graph

`create_hybrid_agent` tạo `react_executor` bằng `create_agent` trước khi khai báo các node của `StateGraph`. Sub-agent dùng cùng model, cùng harness proxy tools và cùng middleware. Khác P&E thuần, một lần chạy node `react_execute` có thể chứa nhiều vòng model tool observation cho đến khi sub-agent tự trả final answer.

7.2 Planner và mục tiêu cấp cao

planner của hybrid dùng `HybridPlan`. Prompt yêu cầu chỉ tạo mục tiêu có thể thực hiện bằng tool thật và đưa trực tiếp mô tả cùng schema của từng tool vào ngữ cảnh. Mục đích là tránh các bước không có action tương ứng như mở giao diện hoặc truy cập database.

7.3 Thu thập observation

Replanner không chỉ nhận câu kết luận của sub-agent. Node `react_execute` gom tất cả `ToolMessage` vào `tool_observations` và ghép với `final_answer`. Nhờ đó quyết định sửa plan có căn cứ từ output tool thật.

```

observation = {
    "tool_observations": [...],
    "final_answer": "...
}

```

7.4 Replanner của hybrid

HybridReplanDecision cũng có action continue hoặc finish. Nếu executor đã hoàn thành mục tiêu, replanner bỏ bước đó hoặc kết thúc toàn bộ yêu cầu. Nếu raw observation báo không có chuyển, tool error hoặc sai arguments, replanner tạo mục tiêu mới cho ReAct executor. Nếu observation là approval_required hoặc executor đang hỏi người dùng, replanner bắt buộc finish để graph không tự lặp trong lúc thiếu input ngoài hệ thống.

8 So sánh ba mẫu thiết kế

Tiêu chí	ReAct	Plan and Execute	Hybrid
Đơn vị quyết định	Một bước kế tiếp	Plan trước, một tool mỗi bước	Mục tiêu cấp cao và ReAct bên trong
Khả năng thích nghi	Cao	Trung bình	Cao
Khả năng theo dõi	Trung bình	Rõ nhờ plan và past_steps	Rõ nhưng phức tạp hơn
Nguy cơ lặp	Cao hơn	Có thể lặp ở replanner	Có thể lặp ở cả hai tầng
Chi phí model	Thường thấp hơn	Nhiều lượt planner executor replanner	Cao nhất do nested ReAct
Use case phù hợp	Tác vụ ngắn linh hoạt	Workflow nhiều bước rõ	Tác vụ phức tạp cần cả plan và phản ứng

9 FlightHarness và chính sách an toàn

FlightHarness là policy layer độc lập với model và graph. Agent chỉ thấy hai StructuredTool có tên quen thuộc, nhưng các tool này trỏ đến method của harness. Nhờ vậy cùng một chính sách được tái sử dụng trong cả ba kiến trúc.

Hàm trong harness	Input chính	Trách nhiệm và output
search_flight_info	departure arrival date hour	Chạy raw search, lưu last_search_results, ghi observation status ok.
propose_booking	airline route date hour	Xác minh chuyến đã search và available; chỉ tạo pending_booking.
submit_booking	approved	Reject hoặc gọi raw book tool; đây là điểm side effect duy nhất.
_guard	tool name arguments	Kiểm tra stop_reason, tool budget và fingerprint lặp.

Hàm trong harness	Input chính	Trách nhiệm và output
_record	action observation	Nối bản ghi vào history rồi trả observation.
langchain_tools	không có	Tạo hai StructuredTool proxy cho agent.
report	không có	Trả stop reason, pending booking, call count và history.

9.1 Ghi observation

Mỗi call được lưu thành bản ghi gồm tool, arguments và observation. report trả stop_reason, pending_booking, tool_call_count và history. Đây là nguồn dữ liệu cho debug và benchmark.

9.2 Xác minh search trước booking

propose_booking chỉ chấp nhận chuyến khớp Airline, Departure, Arrival, Date và Hour trong last_search_results, đồng thời State phải là available. Nếu không khớp, harness trả flight_not_verified và không tạo pending booking.

9.3 Approval gate

Khi agent đề xuất book_flight, propose_booking so sánh arguments với từng bản ghi trong last_search_results. Chỉ bản ghi khớp đủ hãng, điểm đi, điểm đến, ngày, giờ và State available mới được lưu thành pending_booking. Kết quả trả cho agent là status approval_required. Nếu agent gửi lại đúng booking trong lúc đang chờ, harness tiếp tục trả approval_required và không gọi raw tool.

Sau khi graph dừng, chương trình mới đọc câu trả lời của người dùng. submit_booking False ghi observation rejected và stop_reason user_rejected_booking. submit_booking True lấy pending arguments, xóa pending, gọi self._book_tool và đặt stop_reason booking_completed khi raw tool trả success. Vì submit_booking là điểm duy nhất gọi raw book tool nên model không thể tự tạo side effect.

9.4 Phát hiện lặp và ngân sách

Harness tạo fingerprint bằng JSON đã sort key của tool name và arguments. Nếu fingerprint xuất hiện đủ repeat_limit trong cửa sổ loop_window, stop_reason trở thành loop_detected. Nếu history đạt max_tool_calls, harness dừng với tool_budget_exceeded.

Cấu hình	Giá trị benchmark	Ý nghĩa
max_tool_calls	10	Trần số action và user submit được ghi.
loop_window	6	Chỉ xét sáu action gần nhất.
repeat_limit	2	Lặp đúng action lần thứ hai thì dừng.
recursion_limit	30 hoặc 40	Trần cứng cuối cùng của LangGraph.

9.5 HarnessGuardMiddleware

Middleware chạy after_model. Nếu model còn đề xuất tool trong lúc pending_booking đang chờ người dùng, middleware nhảy thẳng đến end và thêm AIMessage yêu cầu xác nhận. Nếu harness đã có stop_reason, middleware cũng ngăn tool call mới.

10 Luồng user submit

1. Agent gọi search proxy; harness chạy search thật và lưu kết quả.
2. Agent chọn một chuyến rồi gọi book_flight proxy.
3. Harness xác minh chuyến và trả approval_required; chưa có side effect.
4. Agent hoặc middleware kết thúc phiên với yêu cầu xác nhận.
5. Chương trình đọc y hoặc n từ người dùng.
6. Chỉ khi người dùng đồng ý, submit_booking True mới gọi raw book_flight.
7. Harness ghi observation cuối và đặt stop_reason booking_completed hoặc user_rejected_booking.

11 Benchmark trong lab03.py

Benchmark giữ model, dữ liệu, tool và harness giống nhau; biến độc lập là design. Mỗi lần chạy tạo harness mới để lịch sử, pending booking và stop reason không rò giữa các agent.

Metric	Cách tính
outcome	booked, safe_stop, loop_stopped hoặc exception.
passed	Book khi scenario yêu cầu book; không book và không exception khi yêu cầu safe_stop.
tool_calls	Số bản ghi trong harness history.

Metric	Cách tính
errors	Số observation có status error hoặc stopped.
loop_detected	stop_reason bắt đầu bằng loop_detected.
approval_requested	History có observation approval_required.
elapsed_seconds	Thời gian thực thi bằng time.perf_counter.

11.1 Cách chạy

```
python lab03.py --design all --scenario all --approve yes
python lab03.py --design react --scenario co-ban
python lab03.py --design pe --scenario chon-loc
python lab03.py --design hybrid --scenario khong-co-ngay
python lab03.py --approve no
```

Chạy toàn bộ tạo chín phiên model nên có thể phát sinh thời gian và chi phí API. Tùy chọn approve mô phỏng user submit tại approval gate để kết quả giữa các design có thể so sánh tự động.

12 Kết quả kiểm thử

Benchmark model thật được chạy ngày 29/09/2026 bằng lệnh `python lab03.py --design all --scenario all --approve yes`. Cả ba thiết kế dùng cùng model, dữ liệu, tool, FlightHarness và cơ chế tự động phê duyệt, nên khác biệt chủ yếu đến từ cách orchestration. Tổng cộng có chín lượt chạy: ba thiết kế nhân với ba scenario.

Design	Scenario	Expected	Outcome	Pass	Tools	Err	Loop	Giây
ReAct	co-ban	book	booked	True	3	0	False	11.9
P&E	co-ban	book	booked	True	4	0	False	75.2
Hybrid	co-ban	book	booked	True	3	0	False	38.8
ReAct	chon-loc	book	booked	True	3	0	False	12.4
P&E	chon-loc	book	booked	True	4	0	False	94.3
Hybrid	chon-loc	book	booked	True	3	0	False	33.9
ReAct	khong-co-ngay	safe_stop	safe_stop	True	1	0	False	6.0
P&E	khong-co-ngay	safe_stop	safe_stop	True	0	0	False	13.3
Hybrid	khong-co-ngay	safe_stop	safe_stop	True	1	0	False	17.1

Kết quả đạt 9/9 lượt. Ở hai scenario đặt vé, cả ba thiết kế đều bỏ qua chuyển rẻ nhưng đã full, chọn đúng chuyển rẻ nhất còn chỗ, dừng tại approval gate rồi chỉ gọi raw_book_flight sau user submit. Ở scenario không có đúng ngày, cả ba đều safe_stop, không tự đổi ngày và không đặt vé.

ReAct nhanh nhất trong cả ba scenario: 11,9 giây, 12,4 giây và 6,0 giây. P&E chậm nhất ở hai tác vụ đặt vé với 75,2 giây và 94,3 giây vì planner, executor và replanner tạo nhiều lượt gọi model. Hybrid nằm giữa ở hai tác vụ đặt vé với 38,8 giây và 33,9 giây; ở tác vụ không có ngày, Hybrid mất 17,1 giây, cao hơn P&E 13,3 giây và ReAct 6,0 giây.

Số tool call cũng phản ánh kiến trúc: ReAct và Hybrid dùng ba call ở mỗi scenario đặt vé, còn P&E dùng bốn call do một bước điều phối bổ sung. Riêng P&E ở scenario không có ngày ghi nhận 0 tool call vì planner hoặc executor kết thúc an toàn trước khi harness nhận lời gọi tool. Đây vẫn là pass theo tiêu chí safe_stop, nhưng response nhắc tool_search_tickets không khả dụng cho thấy prompt hoặc ánh xạ tên tool của P&E còn cần chuẩn hóa.

Loop detector cũng được kiểm tra độc lập: lần search thứ nhất trả ok; lần thứ hai với cùng arguments trả stopped và stop_reason loop_detected. Approval được kiểm tra độc lập: trước submit, pending_booking tồn tại nhưng raw_book_tool chưa chạy; sau submit True, kết quả success và stop_reason là booking_completed.

13 Nhận xét về hiệu quả

13.1 ReAct

ReAct phù hợp khi yêu cầu ngắn và observation của tool đủ rõ. Chi phí orchestration thấp nhưng hành vi khó dự đoán hơn. Harness giúp biến các lỗi như retry và booking sớm thành trạng thái có kiểm soát.

13.2 Plan and Execute

P&E tạo trace dễ giải thích nhất vì mỗi observation gắn với một step. Replanner có thể sửa sai tên tool hoặc arguments dựa trên lỗi Pydantic. Đổi lại, mỗi bước cần nhiều lượt gọi model nên độ trễ cao hơn.

13.3 Hybrid

Hybrid mạnh nhất khi một mục tiêu cấp cao cần nhiều hành động thích nghi. Tuy nhiên nó cũng phức tạp và tốn lượt model nhất. Nếu planner tạo mục tiêu không ánh xạ được sang tool thật, ReAct executor có thể trả lời bằng text thay vì tiến triển; prompt tool schema và loop guard là bắt buộc.

13.4 Bảng so sánh tổng hợp cuối cùng

Nội dung	ReAct	Plan and Execute	Hybrid
Luồng điều phối	Model và tool lập từng bước	Planner rồi executor một tool rồi replanner	Planner rồi ReAct sub-agent rồi replanner
State chính	messages	user_request plan past_steps response	user_request plan past_steps response
Sửa sai sau observation	Model tự phản ứng vòng kế	Replanner sửa remaining_steps	ReAct sửa cục bộ và replanner sửa mục tiêu
Thực thi tool	Harness proxy qua	Harness proxy qua executor	Harness proxy trong ReAct

Nội dung	ReAct	Plan and Execute	Hybrid
	create_agent	tool.invoke	executor
Approval	Middleware hoặc model dừng	Replanner finish khi approval_required	Middleware chặn sub-agent và replanner finish
Phát hiện lặp	FlightHarness và middleware	FlightHarness và recursion limit	FlightHarness middleware và recursion limit
Thời gian trung bình	10,1 giây	60,9 giây	29,9 giây
Tổng tool calls	7	8	7
Kết quả benchmark	3/3 pass	3/3 pass	3/3 pass
Điểm mạnh	Gọn và linh hoạt	Trace rõ để giải thích	Vừa có chiến lược vừa thích nghi
Điểm yếu	Dễ hành động cục bộ và lặp	Nhiều lượt model và phụ thuộc plan	Phức tạp và có nguy cơ lặp hai tầng
Trạng thái kiểm thử	Đã chạy đủ ba scenario	Đã chạy đủ ba scenario	Đã chạy đủ ba scenario

14 Hạn chế và hướng phát triển

- Database hiện là list trong memory, chưa có transaction, concurrency control hoặc persistence.
- book_flight chưa sinh booking ID và chưa quản lý thông tin hành khách.
- Fingerprint exact-match chưa phát hiện được semantic loop khi model đổi cách viết arguments.
- Benchmark mới ghi thời gian và tool calls, chưa đo token usage hoặc chi phí tiền theo provider.
- User approval trong CLI là y hoặc n; sản phẩm thật nên dùng LangGraph interrupt và checkpoint để resume an toàn.
- Structured JSON parser đang trích object bằng biểu thức chính quy; production nên dùng native structured output khi endpoint hỗ trợ ổn định.

Các hướng nâng cấp ưu tiên là thêm checkpointer cho human in the loop, chuẩn hóa dữ liệu bằng Pydantic ở boundary tool, thêm semantic loop detector dựa trên mục tiêu và tiến triển, đồng thời lưu trace benchmark thành JSON hoặc CSV để phân tích nhiều lần chạy.

15 Kết luận

Bài tập đã đi từ một agent có khả năng gọi tool đến một hệ thống agent có orchestration, observation, replanning, approval và đánh giá. ReAct, P&E và Hybrid khác nhau ở cách phân chia quyết định, nhưng cùng dùng một FlightHarness để bảo đảm chính sách nghiệp vụ không phụ thuộc vào sự tự giác của model. Kết quả quan trọng nhất là quyền thực hiện booking đã được chuyển khỏi agent: model chỉ đề xuất, harness xác minh, người dùng phê duyệt và raw tool mới thực thi.

Phụ lục A Lệnh cài đặt và chạy

```
python -m pip install -r requirements.txt
python lib/model_react.py
python lib/model_pe.py
python lib/model_hybrid.py
python lab03.py --design all --scenario all --approve yes
```

Phụ lục B Từ khóa chính

Thuật ngữ	Giải thích ngắn
Action	Tool call mà model đề xuất.
Observation	Kết quả tool được đưa lại vào state hoặc messages.
Replanner	Node đánh giá tiến triển và sửa phần plan còn lại.
Harness	Lớp policy và execution độc lập với model.
Approval gate	Điểm dừng bắt buộc trước side effect.
Middleware	Hook can thiệp vào vòng agent ở các thời điểm xác định.
CompiledStateGraph	Graph đã compile, có invoke, stream và ainvoke.